

CSE471: System Analysis and Design

Project Report

Project Title: SchemeConnect

Mypart

Table of Contents

1. System Request
 - Business need:
 - Business requirements:
 - Business value:
 - Special issues or constraints:
2. Functional Requirements
3. Technology (Framework, Languages)
4. Backend Development
5. User Interface Design
6. Frontend Development
7. User Manual
8. Performance and Network Analysis
9. Github Repo [Public] Link
10. Link of Deployed Project
11. Individual Contribution
12. References

System Request

Business need:

SchemeConnect addresses the administrative complexity of managing Bangladesh government welfare and subsidy programs by creating a single web platform for: eligibility assessment, document management, verification workflows, budget/fund disbursement, and audit/governance oversight. It reduces manual coordination, prevents duplicated benefits through AI-assisted checks, and improves transparency via ledger/audit logs.

Business requirements:

- Support role-based access for Citizens, Verification Officers, Finance Officers, Administrators, and Auditors.
- Provide structured eligibility profiling so citizens don't repeatedly enter the same data.
- Enable digital document storage and reusability across applications.
- Offer verification case workspace with investigation notes, status updates, and attachments.
- Include AI fraud/duplicate detection producing explanations for officers (human decision remains final).
- Provide budget allocation, ledger, and disbursement workflow using Stripe (or your chosen payment approach).
- Maintain governance audit center logging all critical actions.
- Sync official policy/circular updates from Google Drive into scheme rules.

Business value:

1. Reduced administrative overhead by replacing spreadsheets/email workflows with centralized case management.
2. Improved transparency via audit logs and financial ledger trails.
3. Faster processing through dashboards for workload, overdue inspections, and performance analytics.
4. Better compliance through policy synchronization and audit reporting.
5. Fraud/duplicate reduction using AI-assisted anomaly explanations.

Special issues or constraints:

1. Data privacy & security for national ID and personal documents (must use secure storage patterns).
2. Human-in-the-loop requirement: AI risk suggestions cannot automatically deny approvals.
3. Accuracy of policy synchronization: schema eligibility must reflect the latest circulars.
4. Interoperability with external APIs: Twilio, Stripe, Grok AI, Google Maps, Google Drive.
5. Scalability: platform should remain responsive even with many applications and audit logs.

Functional Requirements

User Registration & Authentication

Citizens register with full name, NID, email, contact, password, district, and division.

Officers/admins are provisioned by administrators.

Role-based login routes to appropriate dashboards.

Role-Based Access Control (RBAC) & Administration

Admin assigns roles and controls access for modules and sensitive actions.

Admin can activate/suspend/deactivate accounts.

Citizen Eligibility Profiler

The questionnaire captures income, occupation, disability status, education, family size, district, etc. System generates and stores standardized eligibility profiles.

AI Fraud & Duplicate Detection

Detect suspicious similarities, conflicts, unusual patterns, or duplicates.

Produces risk explanation for officers.

The final decision remains human.

Welfare Scheme Configuration Studio

Admin creates schemes, manages budgets, tracks allocations, and views ledger.

Finance/admin controls approved financial operations.

Beneficiary Lifecycle Management

Track beneficiary records, decisions, and scheme performance analytics.

Maintain audit logs.

Sync official documents/policy updates for compliance.

Digital Welfare Document Vault

Citizens upload documents into a secure centralized vault.

Verified documents become reusable for future applications.

Verification Case Workspace

Each application gets a structured case assigned to verification officers.

Officers record findings, request clarifications, attach evidence, and update status.

Budget Allocation Intelligence

Track allocated budgets, utilized funds, remaining balances.

District-wise and scheme-wise expenditure analytics.

Low-budget alerts and utilization trends.

Welfare Performance Intelligence

Dashboards for total applications, approval/rejection rates, district distribution, processing time, scheme popularity.

Filters by scheme/district/time/beneficiary category.

Welfare Opportunity Explorer

Scheme directory with eligibility criteria, deadlines, required documents, benefits, and status.

Cross-references citizen's eligibility profile to highlight matching schemes.

Nearby Welfare Office Navigator

Map-based search using Google Maps.

Shows office locations and navigation directions.

Field Inspection Assignment Center

Assign inspection tasks, deadlines, outcomes, evidence.

Track overdue/remaining assignments.

Verification Operations Dashboard

Pending/overdue cases, officer performance metrics.

Twilio SMS reminders for approaching inspection deadlines.

Direct Benefit Disbursement

Finance officers initiate disbursement via bKash/Stripe integration (as implemented).

System verifies transaction, stores reference, updates status, generates digital receipt.

Governance Audit Center

Tamper-resistant logs for approvals, rejections, updates, budget changes, disbursements, and configuration actions.

Search/filter audit records for audits and compliance.

Official Circular Synchronization

Google Drive sync for eligibility criteria, policy documents, and operational guidelines.

Automatic update availability to officers/citizens.

Technology (Framework, Languages)

Frontend

Vanilla.js + TypeScript
TailwindCSS
UI routing for role-based dashboards
Map UI (Google Maps integration)

Backend

Node.js + Express.js
TypeScript
MongoDB database
Mongoose ODM

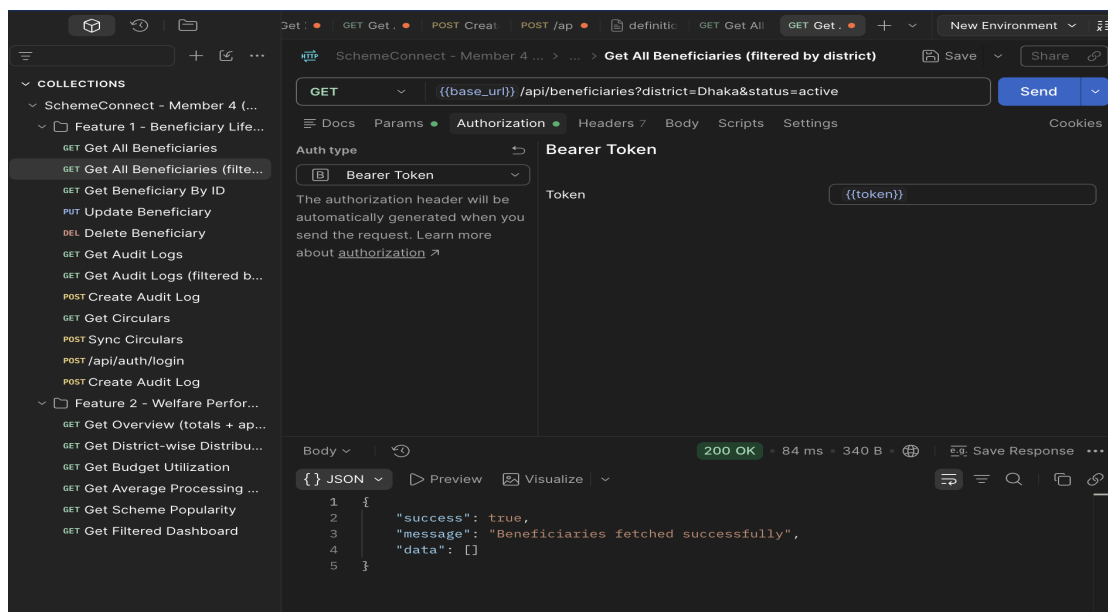
External APIs

- **Grok API:** For AI-assisted fraud detection and duplicate application analysis
- **Stripe API:** For processing direct benefit disbursements to approved beneficiaries
- **Google Maps API :** For displaying nearby welfare offices, union digital centers, and service locations to citizens.
- **Google Drive API :** For synchronizing official government circulars, policy updates, and application forms into the platform.

Backend Development

1. This is a Node.js/Express controller file `beneficiaryController.js` that handles CRUD operations for "beneficiary" records in what appears to be an aid/welfare scheme management system. It includes three handlers: `getAllBeneficiaries` (lists beneficiaries with optional region/status/schemeId filters), `createBeneficiary` (registers a new beneficiary and logs the action to an audit trail), and `getBeneficiaryById` (fetches a single beneficiary by ID, returning a 404 if not found). It uses helper utilities for async error handling, standardized error creation, and consistent success responses. Here, `updateBeneficiary` finds a beneficiary by ID, merges in the request body fields via `Object.assign`, saves the updated record, and logs a `BENEFICIARY_UPDATED` audit entry. `deleteBeneficiary` — finds and deletes a beneficiary by ID, then logs a `BENEFICIARY_DELETED` audit entry.

```
src > controllers > JS beneficiaryController.js > ...
1  const Beneficiary = require('../models/Beneficiary');
2  const AuditLog = require('../models/AuditLog');
3  const asyncHandler = require('../utils/asyncHandler');
4  const createError = require('../utils/appError');
5  const { sendSuccess } = require('../utils/apiResponse');
6  const ROLES = require('../constants/roles');
7
8  // GET all beneficiaries (supports ?region=&status=&schemeId= filters)
9  exports.getAllBeneficiaries = asyncHandler(async (req, res) => {
10     //if (req.user.role !== 'Administrator')
11     //  throw createError('Forbidden', 403);
12     const { region, status, schemeId } = req.query;
13     const filter = {};
14     if (region) filter.region = region;
15     if (status) filter.status = status;
16     if (schemeId) filter.schemeId = schemeId;
17
18     const beneficiaries = await Beneficiary.find(filter).populate('schemeId', 'name');
19     return sendSuccess(res, 200, "Beneficiaries fetched successfully", beneficiaries);
20 });
21
22 // POST register a new beneficiary + logs the action
23 exports.createBeneficiary = asyncHandler(async (req, res) => {
24     const beneficiary = await Beneficiary.create(req.body);
25
26     await AuditLog.create({
27       action: 'BENEFICIARY_REGISTERED',
28       performedBy: req.body.registeredBy || ROLES.ADMINISTRATOR,
29       role: ROLES.ADMINISTRATOR,
30       targetType: 'Beneficiary',
31       targetId: beneficiary._id.toString(),
32       details: `New beneficiary registered: ${beneficiary.name} (${beneficiary.phone})`,
33     });
34
35     return sendSuccess(res, 201, "Beneficiary registered successfully", beneficiary);
36 });
37
38 // GET single beneficiary by id
39 exports.getBeneficiaryById = asyncHandler(async (req, res) => {
40     const beneficiary = await Beneficiary.findById(req.params.id).populate('schemeId', 'name');
41     if (!beneficiary) {
42       throw createError('Beneficiary not found', 404);
43     }
44     return sendSuccess(res, 200, "Beneficiary fetched successfully", beneficiary);
45 });
```



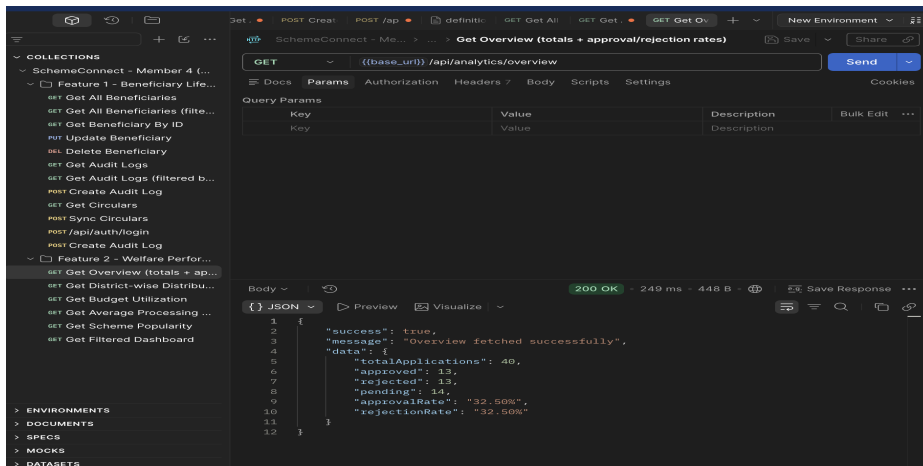
2. This is an analytics controller (analyticsController.js) providing aggregated reporting endpoints for the SchemeConnect application. It includes: `getOverview` total application count with approved/rejected/pending breakdown and approval/rejection rates. `getRegionDistribution` where beneficiary counts grouped by region, sorted descending. `getBudgetUtilization` per-scheme budget allocated vs. utilized, remaining budget, and utilization rate. `getProcessingTime` average number of days between application submission and decision.

`getSchemePopularity` — schemes ranked by number of applications received. `getDashboard` a filterable view of applications (by `schemeId`, `district`, `category`, `date range`) with status breakdown. `getSchemeAnalytics` computing: `activeRate` (% of beneficiaries not flagged), `avgVerificationTimeDays` (average time from case opened to verified), `complianceScore` (a weighted 100-point score penalizing flagged beneficiaries, overdue verifications, and failed disbursements)

```

src > controllers > JS analyticsController.js > ...
1  const Application = require('../models/Application');
2  const Scheme = require('../models/Scheme');
3  const Beneficiary = require('../models/Beneficiary');
4  const asyncHandler = require('../utils/asyncHandler');
5  const createError = require('../utils/appError');
6  const { sendSuccess } = require('../utils/apiResponse');
7
8  // GET total applications, approval/rejection rates
9  exports.getOverview = asyncHandler(async (req, res) => {
10     const total = await Application.countDocuments();
11     const approved = await Application.countDocuments({ status: 'appro
12     const rejected = await Application.countDocuments({ status: 'rejec
13     const pending = await Application.countDocuments({ status: 'pendin
14
15     return sendSuccess(res, 200, "Overview fetched successfully", {
16         totalApplications: total,
17         approved,
18         rejected,
19         pending,
20         approvalRate: total ? ((approved / total) * 100).toFixed(2) +
21         rejectionRate: total ? ((rejected / total) * 100).toFixed(2) +
22     });
23 });
24
25 // GET region-wise beneficiary distribution
26 exports.getRegionDistribution = asyncHandler(async (req, res) => {
27     const data = await Beneficiary.aggregate([
28         { $group: { _id: '$region', totalBeneficiaries: { $sum: 1 } } },
29         { $project: { _id: 0, region: '$_id', totalBeneficiaries: 1 } },
30         { $sort: { totalBeneficiaries: -1 } }
31     ]);
32     return sendSuccess(res, 200, "Region distribution fetched successfully", d
33 });
34
35 // GET budget utilization (allocated vs utilized) per scheme
36 exports.getBudgetUtilization = asyncHandler(async (req, res) => {
37     const schemes = await Scheme.find({}, 'name budgetAllocated budgetUtilized
38     const data = schemes.map((s) => ({
39         scheme: s.name,
40         budgetAllocated: s.budgetAllocated,
41         budgetUtilized: s.budgetUtilized,
42         remaining: s.budgetAllocated - s.budgetUtilized,
43         utilizationRate: s.budgetAllocated
44             ? ((s.budgetUtilized / s.budgetAllocated) * 100).toFixed(2) + '%'

```

3.This is a controller file (`auditLogController.js`) for managing audit trail entries in the SchemeConnect system. It includes two handlers:`getAuditLogs` — fetches audit logs with optional filtering by role, action, and a timestamp date range (`startDate/endDate`), sorted newest-first.`createAuditLog` — creates a new audit log entry directly from the raw request body, intended for manual logging of actions from other modules that don't already handle their own audit logging inline

```

src > controllers > JS auditLogController.js > ...
1  const AuditLog = require('../models/AuditLog');
2  const asyncHandler = require('../utils/asyncHandler');
3  const { sendSuccess } = require('../utils/apiResponse');
4
5  // GET audit logs (supports ?role=&action=&startDate=&endDate= filters)
6  exports.getAuditLogs = asyncHandler(async (req, res) => {
7    const { role, action, startDate, endDate } = req.query;
8    const filter = {};
9    if (role) filter.role = role;
10   if (action) filter.action = action;
11   if (startDate || endDate) {
12     filter.timestamp = {};
13     if (startDate) filter.timestamp.$gte = new Date(startDate);
14     if (endDate) filter.timestamp.$lte = new Date(endDate);
15   }
16
17   const logs = await AuditLog.find(filter).sort({ timestamp: -1 });
18   return sendSuccess(res, 200, "Audit logs fetched successfully", logs);
19 });
20
21 // POST create a new audit log entry manually (e.g. for actions from other modules)
22 exports.createAuditLog = asyncHandler(async (req, res) => {
23   const log = await AuditLog.create(req.body);
24   return sendSuccess(res, 201, "Audit log created successfully", log);
25 });
26

```

The screenshot shows a REST client interface with the following details:

- Endpoint:** GET `{{base_url}}/api/audit-logs`
- Method:** GET
- Status:** 200 OK, 135 ms, 18.83 KB
- Response Body (JSON):**

```

{
  "success": true,
  "message": "Audit logs fetched successfully",
  "data": [
    {
      "_id": "6a98f159475f62899764365d",
      "action": "CIRCULARS_SYNCED",
      "performedBy": "admin@schemeconnect.test",
      "role": "Administrator",
      "targetType": "Circular",
      "details": "1 circular(s) synced from Google Drive",
      "timestamp": "2026-09-03T04:02:33.005Z",
      "__v": 0
    },
    {
      "_id": "6a93cd8f6a4c56ec90fac6eb",
      "action": "CIRCULARS_SYNCED",

```

4.This is a controller file (circularController.js) for managing circulars mostly official government notices/documents synced from Google Drive.It includes three handlers:getCirculars — fetches all circulars from the database, sorted by publishedDate in descending order (newest first).syncCirculars — simulates pulling new circulars from Google Drive by accepting an array of circular objects (title, description, fileUrl, publishedDate) in the request body with an added syncedAt timestamp, and logging a CIRCULARS_SYNCED audit entry. Returns a 400 error if the array is missing or empty.deleteCircular — deletes a circular by ID, returning a 404 if not found, and logs a CIRCULAR_DELETED audit entry including the circular's title.

```

const Circular = require('../models/Circular');
const AuditLog = require('../models/AuditLog');
const asyncHandler = require('../utils/asyncHandler');
const createError = require('../utils/appError');
const { sendSuccess } = require('../utils/apiResponse');

// GET all synced circulars
exports.getCirculars = asyncHandler(async (req, res) => {
  const circulars = await Circular.find().sort({ publishedDate: -1 });
  return sendSuccess(res, 200, "Circulars fetched successfully", circulars);
});

// POST sync circulars (simulates pulling new circulars from Google Drive)
exports.syncCirculars = asyncHandler(async (req, res) => {
  const { circulars } = req.body; // expects: [{ title, description, fileUrl, ... }]
  if (!circulars || !Array.isArray(circulars) || circulars.length === 0) {
    throw createError('A non-empty "circulars" array is required in the request body');
  }

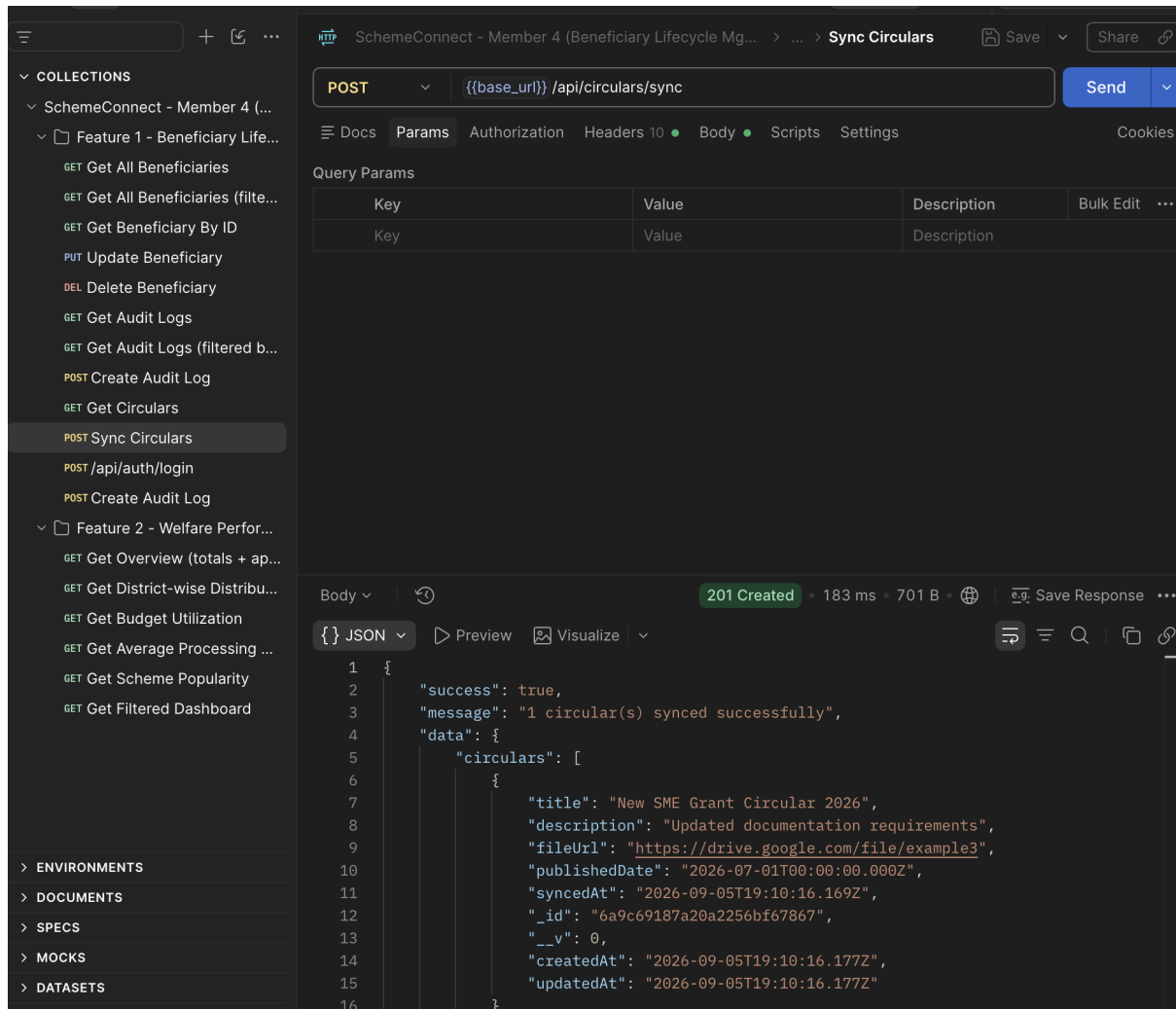
  const inserted = await Circular.insertMany(
    circulars.map((c) => ({
      ...c,
      syncedAt: new Date()
    })))
  );

  await AuditLog.create({
    action: 'CIRCULARS_SYNCED',
    performedBy: req.user.email,
    role: req.user.role,
    targetType: 'Circular',
    details: `${inserted.length} circular(s) synced from Google Drive`
  });

  return sendSuccess(res, 201, `${inserted.length} circular(s) synced successfully`, {
    circulars: inserted
  });
});

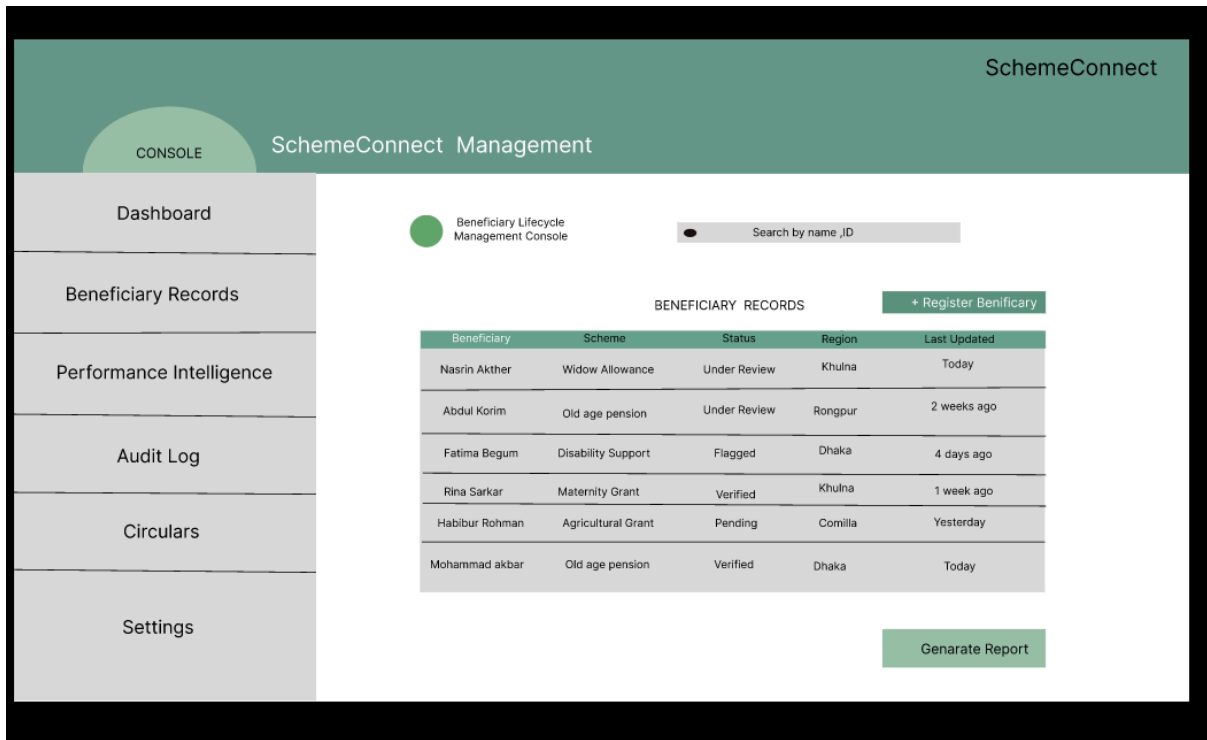
// DELETE a circular + logs the action
exports.deleteCircular = asyncHandler(async (req, res) => {
  const deleted = await Circular.findByIdAndDelete(req.params.id);
  if (!deleted) {
    throw createError('Circular not found', 404);
  }
});

```



Beneficiary Lifecycle Management: This feature provides administrative oversight and decision-support tools for government authorities. It helps monitor beneficiary records, evaluate scheme performance through analytics, maintain comprehensive audit logs, and synchronize official government circulars to ensure policy compliance and informed decision-making.

Figma Screenshots related to Beneficiary Lifecycle Management



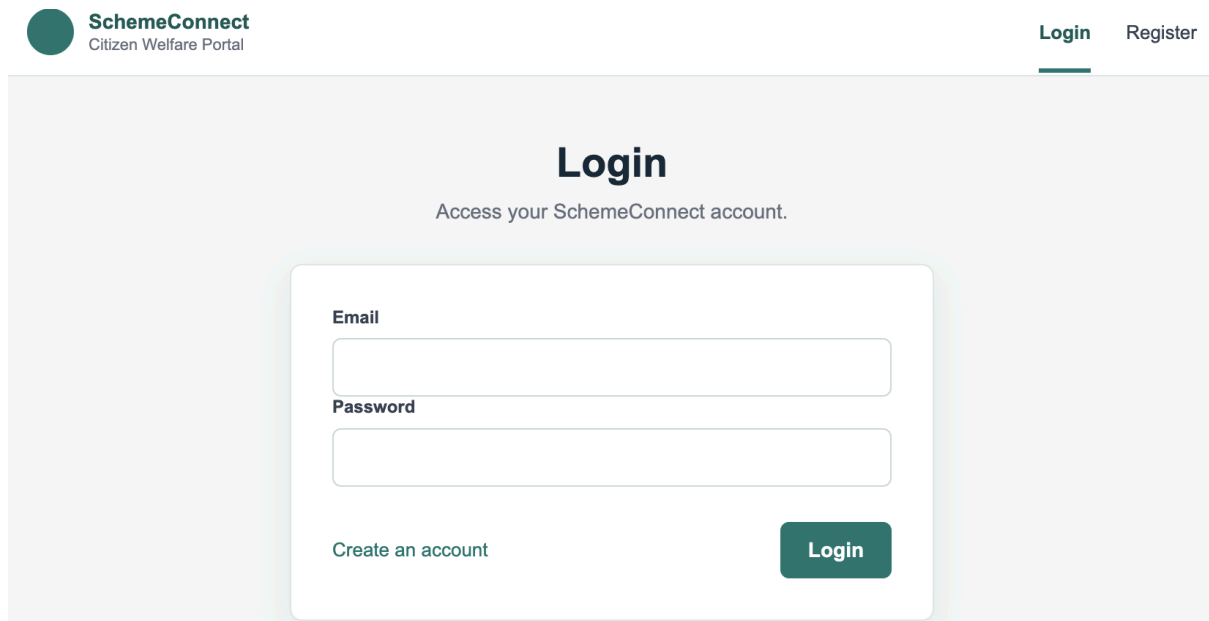
Figma Prototype Link

<https://www.figma.com/proto/rNrt2cSDLVb0unqepAbBVh/FARIHA-TASNIM-ALAM-s-team-library?node-id=3318-2&p=f&t=5fpuEwwjuit2FxbO-1&scaling=contain&content-scaling=fixed&page-id=3313%3A2&starting-point-node-id=3318%3A2>

Govt.Administrator:

Logging In

Navigate to the application's login page (/login). Enter the Administrator account's email and password, then click Login. The system checks the account's role. Administrators are routed directly into the Beneficiary Records screen of the Administrator Console. Citizens are routed to a separate citizen-facing dashboard instead.



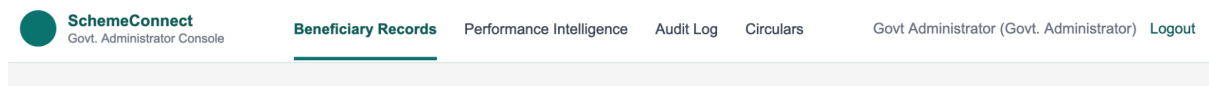
The image shows the login page of the SchemeConnect Citizen Welfare Portal. At the top left is the SchemeConnect logo and text. At the top right are links for 'Login' and 'Register'. The main heading is 'Login' with the subtext 'Access your SchemeConnect account.' Below this is a form with two input fields: 'Email' and 'Password'. At the bottom left of the form is a link 'Create an account' and at the bottom right is a green 'Login' button.

The Administrator Console Layout

Every screen in the console shares a consistent layout:

Top bar — shows the SchemeConnect logo, the console name, the logged-in administrator's name and role, and a Logout button.

Navigation bar :four tabs corresponding to the four features: Beneficiary Records, Performance Intelligence, Audit Log, and Circulars.



The image shows the top bar of the SchemeConnect Govt. Administrator Console. It includes the SchemeConnect logo and text on the left. In the center are four navigation tabs: 'Beneficiary Records' (which is underlined), 'Performance Intelligence', 'Audit Log', and 'Circulars'. On the right, it shows the user's name and role 'Govt Administrator (Govt. Administrator)' and a 'Logout' button.

Beneficiary Lifecycle Management

This screen lets an administrator view, register, update, search, and remove beneficiary records across all welfare schemes. Viewing records

Click the Beneficiary Records tab. A table loads showing every beneficiary's name, linked scheme, status, district, and when the record was last updated.

Search by name or National ID				+ Register Beneficiary	
BENEFICIARY	SCHEME	STATUS	REGION	LAST UPDATED	ACTIONS
Fariha	Women Entrepreneur Grant	Verified	Dhaka	2 days ago	Edit Delete
Mikey Mouse	Farmer Subsidy Programme	Flagged	Chittagong	Today	Edit Delete
Mini Mouse	Women Entrepreneur Grant	Verified	Sylhet	2 days ago	Edit Delete
Tasnim Aktar	Primary Education Stipend	Pending	Rajshahi	2 days ago	Edit Delete
Mamun Miya	Disability Allowance	Verified	Sylhet	2 days ago	Edit Delete
Tom	Maternal Health Support	Verified	Chittagong	2 days ago	Edit Delete
Jerry	Rural Housing Assistance	Verified	Dhaka	Today	Edit Delete
Lisa	Secondary School Scholarship	Flagged	Chittagong	2 days ago	Edit Delete
Lamiya	Secondary School Scholarship	Under Review	Chittagong	2 days ago	Edit Delete
Moona	Rural Housing Assistance	Verified	Khulna	2 days ago	Edit Delete
Tommy	SME Development Loan	Under Review	Rajshahi	2 days ago	Edit Delete
Molla Mia	Disability Allowance	Flagged	Dhaka	2 days ago	Edit Delete
Jennie	Women Entrepreneur Grant	Verified	Dhaka	2 days ago	Edit Delete
Jisso	Maternal Health Support	Under Review	Sylhet	2 days ago	Edit Delete

Searching

Type a name or National ID into the search box above the table. The table filters instantly as you type — no page reload needed.


SchemeConnect
Govt. Administrator Console

[Beneficiary Records](#)
[Performance Intelligence](#)
[Audit Log](#)
[Circulars](#)

Govt Administrator (Govt. Administrator)
[Logout](#)

Beneficiary Records

Register, review, and manage beneficiaries across all welfare schemes.

[+ Register Beneficiary](#)

BENEFICIARY	SCHEME	STATUS	REGION	LAST UPDATED	ACTIONS
Mini Mouse	Women Entrepreneur Grant	Verified	Sylhet	2 days ago	Edit Delete

Registering a new beneficiary

Click + Register Beneficiary.

A popup form appears requesting Full Name, National ID, Contact Number, District, Division, and Scheme ID.

Fill in the fields and click Save Beneficiary.

The popup closes automatically and the table refreshes to show the new record.

The screenshot shows a web application interface. At the top right, there is a dark green button labeled '+ Register Beneficiary'. Below it is a table with columns: NAME, SCHEME, STATUS, REGION, LAST UPDATED, and ACTIONS. The table contains three rows of beneficiary data. A white modal popup titled 'Register Beneficiary' is open in the center. It has a close button (X) in the top right corner. The form inside the modal has five input fields: 'Full name', 'National ID', 'Contact number', 'Region', and 'Scheme ID'. A dark green button labeled 'Save Beneficiary' is at the bottom right of the modal. The background table shows the following data:

NAME	SCHEME	STATUS	REGION	LAST UPDATED	ACTIONS
Women Entrepreneur Grant	Verified	Dhaka	2 days ago	Edit	Delete
Farmer Subsidy Programme	Flagged	Chittagong	Today	Edit	Delete
Rural Housing Assistance	Verified	Khulna	2 days ago	Edit	Delete
SME Development Loan	Under Review	Rajshahi	2 days ago	Edit	Delete

Editing a beneficiary

Click Edit on any row.

A popup opens pre-filled with that beneficiary's current details, including a Status dropdown.

Update any field and click Save Changes.

The screenshot shows the 'Update Beneficiary' modal popup. It has a close button (X) in the top right corner. The form is pre-filled with the following data:

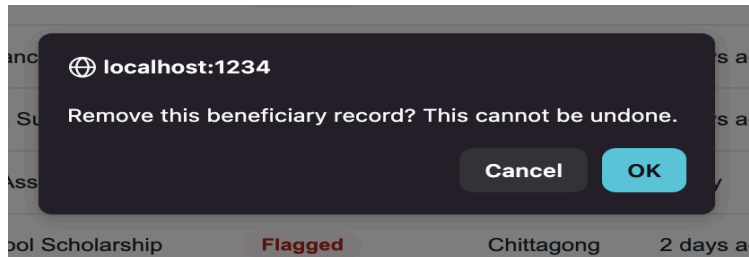
- Full name: Mini Mouse
- National ID: BNID2003
- Contact number: 01912388878
- Region: Sylhet
- Scheme ID: 6a8b3162243c9cc75b85d460
- Status: Verified (dropdown menu)

A dark green button labeled 'Save Changes' is at the bottom center of the modal.

Click Delete on any row.

A confirmation prompt appears to prevent accidental deletion.

Confirming removes the record and refreshes the table.



Welfare Performance Intelligence

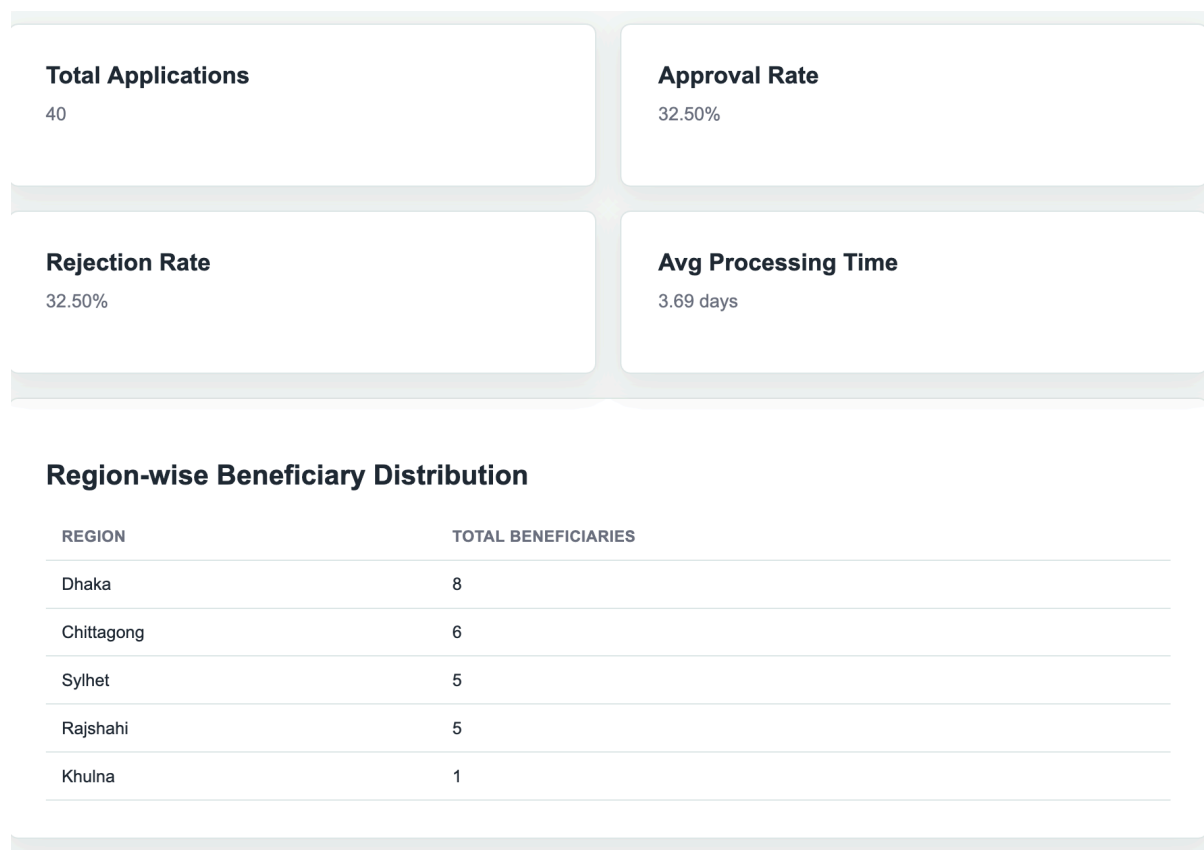
This screen gives administrators a centralized analytics view of scheme performance, with the ability to filter by scheme, district, category, and time period.

Filtering applications

The image shows a web application interface for 'SchemeConnect Govt. Administrator Console'. The top navigation bar includes links for 'Beneficiary Records', 'Performance Intelligence' (which is the active tab), 'Audit Log', and 'Circulars'. On the right side of the navigation bar, it shows the user is 'Govt Administrator (Govt. Administrator)' with a 'Logout' link. The main content area is titled 'Welfare Performance Intelligence' with a subtitle 'Track applications, budgets, and processing performance across schemes.' Below this is a 'Filter Applications' panel. This panel contains four input fields: 'Scheme ID' (with a placeholder 'Optional'), 'District' (with a placeholder 'Optional'), 'Category' (a dropdown menu currently showing 'All categories'), and 'Start date' (a date picker showing 'mm / dd / yyyy'). There is also an 'End date' field with a date picker showing 'mm / dd / yyyy'. An 'Apply Filters' button is located at the bottom right of the filter panel.

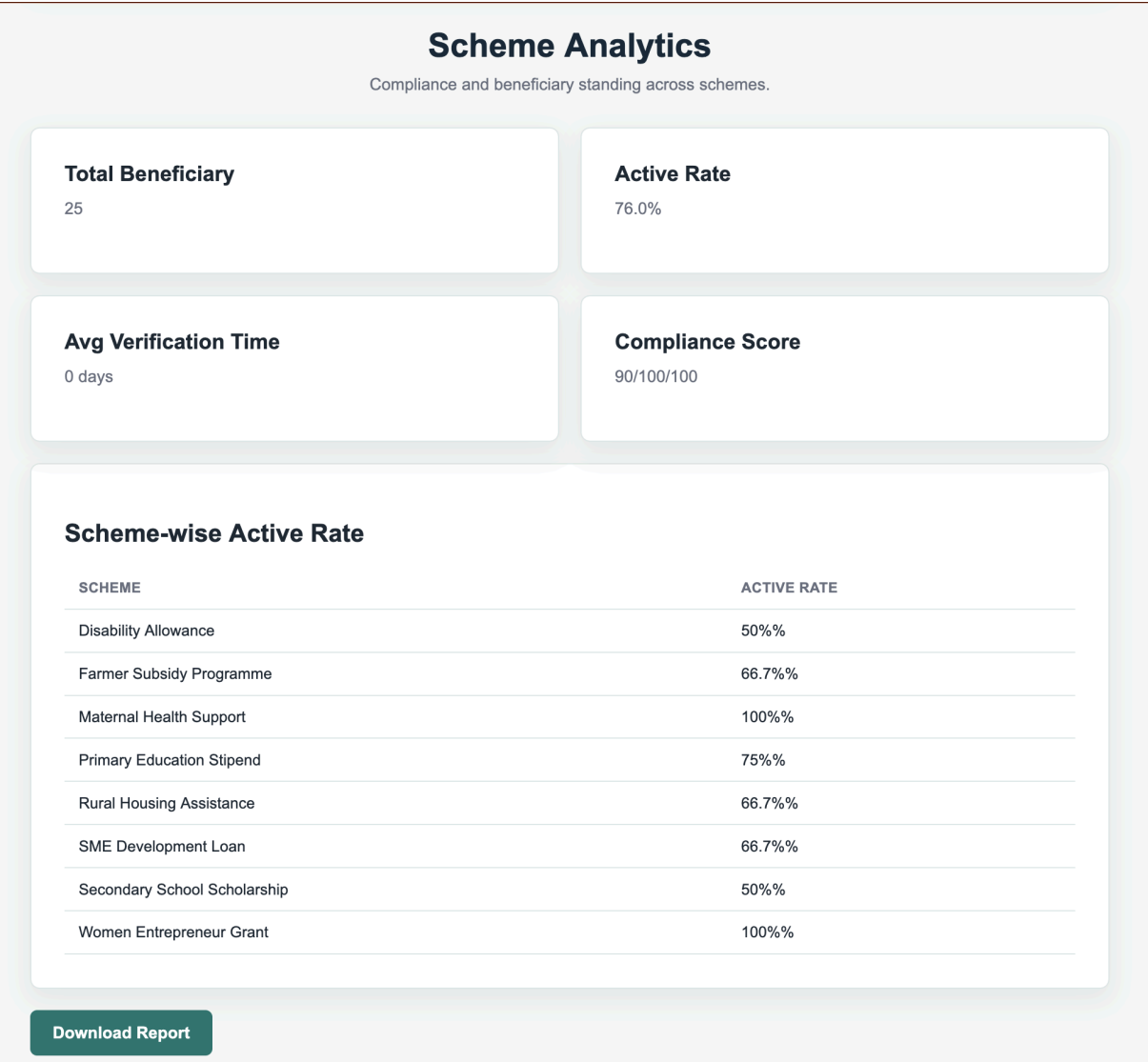
Click the Performance Intelligence tab.

In the "Filter Applications" panel, optionally enter a Scheme ID, District, Category, and/or date range. Click Apply Filters. A "Filtered Results" panel appears showing the matching total applications, approval rate, rejection rate, and a district breakdown.



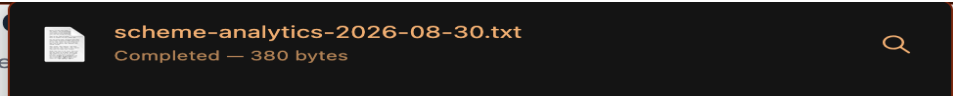
Reviewing platform-wide statistics Below the filtered section, the page shows unfiltered totals for reference: Total Applications, Approval Rate, Rejection Rate, Average Processing Time, a Region-wise Beneficiary Distribution table, a Budget Utilization table per scheme, and a Scheme Popularity ranking.

Scheme Analytics Further down the same page, a "Scheme Analytics" section shows Total Beneficiary count, Active Rate, Average Verification Time, Compliance Score, and a Scheme-wise Active Rate breakdown.



Downloading a report

Click Download Report at the bottom of the page.A .txt file summarizing the Scheme Analytics numbers downloads to the administrator's computer.



Governance Audit Center

This screen provides a read-only compliance trail of every recorded action across the platform.

Reviewing the log

Click the Audit Log tab.
A table loads showing Timestamp, Action, Performed By, Role, Target, and Details for every logged event — for example, a beneficiary being registered, updated, or deleted, or a circular being synced.

Governance Audit Center

Review every recorded action across the platform for compliance.

Filter Logs

Role

All roles

Action

e.g. BENEFICIARY_UPDATED

Start date

mm / dd / yyyy

End date

mm / dd / yyyy

Apply Filters

TIMESTAMP	ACTION	PERFORMED BY	ROLE	TARGET	DETAILS
8/30/2026, 11:21:35 AM	CIRCULARS_SYNCED	admin@schemeconnect.test	Govt. Administrator	Circular	1 circular(s)
8/30/2026, 11:16:57 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:16:32 AM	BENEFICIARY_REGISTERED	Govt. Administrator	Govt. Administrator	Beneficiary	New benefi
8/30/2026, 11:15:25 AM	BENEFICIARY_REGISTERED	Govt. Administrator	Govt. Administrator	Beneficiary	New benefi
8/30/2026, 11:13:59 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:13:51 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:13:42 AM	BENEFICIARY_REGISTERED	Govt. Administrator	Govt. Administrator	Beneficiary	New benefi
8/30/2026, 11:12:26 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:11:45 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:11:00 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:09:39 AM	BENEFICIARY_UPDATED	Govt. Administrator	Govt. Administrator	Beneficiary	Beneficiary
8/30/2026, 11:09:29 AM	BENEFICIARY_REGISTERED	Govt. Administrator	Govt. Administrator	Beneficiary	New benefi
8/30/2026, 11:08:43 AM	BENEFICIARY_REGISTERED	Govt. Administrator	Govt. Administrator	Beneficiary	New benefi
8/28/2026, 3:43:35 AM	CIRCULARS_SYNCED	admin@schemeconnect.test	Govt. Administrator	Circular	1 circular(s)
8/28/2026, 3:23:22 AM	CIRCULARS_SYNCED	admin@schemeconnect.test	Govt. Administrator	Circular	1 circular(s)
8/28/2026, 3:21:26 AM	CIRCULARS_SYNCED	admin@schemeconnect.test	Govt. Administrator	Circular	1 circular(s)

Filtering the log

In the "Filter Logs" panel, optionally choose a Role, type an Action name, and/or set a date range. Click Apply Filters. The table narrows to matching entries only.

Filter Logs

Role

- ✓ All roles
- Citizen
- Verification Officer
- Finance Officer
- Administrator
- Auditor

Official Circular Synchronization

This screen lets administrators sync official policy documents so eligibility rules stay current across all schemes. Viewing synced circulars Click the Circulars tab.

Below the sync form, every previously synced circular is listed with its title, description, and a clickable link to the source document.

**SchemeConnect**
Govt. Administrator Console

Beneficiary Records

Performance Intelligence

Audit Log

Circulars

Govt Administrator (Govt. Administrator) [Logout](#)

Official Circular Synchronization

Synced circulars update eligibility rules across all schemes automatically.

Circular title

Description

Google Drive file URL

Published date

Sync from Google Drive

For Syncing a new circular

Fill in the Circular Title, Description, Google Drive file URL, and Published Date fields. Click Sync from Google Drive. The new circular appears at the top of the list below.

Rural Grant Documentation Requirements

Applicants must now submit income proof

https://docs.google.com/document/d/1u8_aZyA2LJI0I_E0yYA0SDKbH16QeGcM9D8_WKMub-k/edit?usp=sharing

Delete

Healthcare Scheme Documentation Update

New vaccination record requirement for child beneficiaries

<https://drive.google.com/file/d/example-health-2026>

Delete

Updated SME Grant Documentation Requirements

Applicants must now submit trade license

<https://drive.google.com/file/d/example-sme-2026>

Delete

Updated SME Grant Documentation Requirements

Applicants must now submit trade license

<https://drive.google.com/file/d/example-sme-2026>

Delete

Logging Out

Click Logout in the top-right corner of any console screen at any time to end the session and return to the login page.

Logout



\

The system UI must be responsive.

Github Repo [Public] Link

<https://github.com/Rubai106/SchemeConnect>

<https://github.com/Rubai106/SchemeConnect/tree/rubai-dev>

<https://github.com/Rubai106/SchemeConnect/tree/fariha-features>

<https://github.com/Rubai106/SchemeConnect/tree/mahima-features>

<https://github.com/Rubai106/SchemeConnect/tree/nafisa-backend-features>

Link of Deployed Project

<https://schemeconnect-iynj.onrender.com/>

Individual Contribution

Group member - 01	
Name: Samin Yeasar Rubai	Student ID:23201275
Functional Requirements which are developed by this member:	
1.Citizen Eligibility Profiler	
2.Digital Welfare Document Vault	
3.Welfare Opportunity Explorer	
4.Nearby Welfare Office Navigator	

Group member - 02	
Name: Nafisa Hasan	Student ID:22299141
Functional Requirements which are developed by this member:	
1.AI Fraud & Duplicate Detection	

2.Verification Case Workspace
3.Verification Operations Dashboard
4.Field Inspection Assignment Center

Group member - 03	
Name: Mahima Islam	Student ID: 21201143
Functional Requirements which are developed by this member:	
1.Welfare Scheme Configuration Studio	
2.Budget Allocation Intelligence	
3.Financial Transaction Records	
4.Direct Benefit Disbursement	

Group member - 04	
Name: Fariha Tasnim Alam	Student ID:23101451
Functional Requirements which are developed by this member:	
1.Beneficiary Lifecycle Management	
2.Welfare Performance Intelligence	
3.Governance Audit Center	
4.Official Circular Synchronization	